

EHAX Project Details

1. In-Memory Database

- **What to build:** An interactive command-line interface (CLI / REPL) that stores and manipulates key-value data directly in memory.
- **Core Features:**
 - Support commands: `SET <key> <value>`, `GET <key>`, `DEL <key>`, and `EXISTS <key>`.
 - Add basic persistence: `SAVE <filename>` (dumps current state to a file) and `LOAD <filename>` (restores state on startup).
 - Robust command parsing, input validation, and missing-key error handling.
- **Constraints:**
 - Strictly **no database or caching libraries** (no SQLite, TinyDB, Redis-py, etc.).
 - If using C: Implement dynamic memory allocation (`malloc/free`) and data structures from scratch using pointers.
 - If using Python: Use only standard built-ins; do not wrap third-party storage utilities.
- **Brownie Points:**
 - Key expiration support with TTL: `SETEX <key> <seconds> <value>`.
 - Support for multiple data types (e.g., lists with `LPUSH` / `LPOP`).

2.Auth Log Parser

- **What to build:** A CLI utility that ingests system authentication logs (e.g., Linux `/var/log/auth.log` or SSH logs) and extracts structured authentication events.
- **Core Features:**
 - Accept a log file path via command-line arguments.
 - Accurately parse and extract the following fields for each entry:
 - **Timestamp**
 - **Source IP**
 - **Username**
 - **Event type** (e.g., SSH authentication, sudo command execution, session open/close)
 - **Success / Failure** status
 - **PID** (Process ID)
 - **Port**
 - Display an analytical summary: Total login attempts, overall success vs. failure breakdown, top targeted usernames, and top originating source IPs.
 - Gracefully handle and skip malformed or non-auth lines without crashing.
- **Constraints:**
 - Strictly **no data analysis or specialized log-parsing libraries** (no `pandas`, `polars`, `numpy`, etc.).
 - Rely strictly on native file I/O, string manipulation, and standard regular expressions (`re` in Python or standard POSIX regex/string handling in C).
- **Brownie Points:**
 - Brute-force detection heuristic: Automatically flag source IPs that exceed a threshold of consecutive failed login attempts within a given time window.
 - Live monitoring mode that parses incoming auth attempts in real time as they are appended to the file (like `tail -f`).
 - Export parsed structured events into clean JSON or CSV format.

3. Basic Cipher Implementation

- **What to build:** A command-line encryption and decryption tool implementing classical ciphers along with standard **AES (Advanced Encryption Standard)**.
- **Core Features:**
 - Implement at least one classical cipher (e.g., Vigenère Cipher or Rail Fence Cipher) **and** an implementation of **AES-128** from scratch in Electronic Codebook (ECB) mode.
 - Support encrypting (-e) and decrypting (-d) both direct string arguments and plain .txt files.
 - Handle block padding properly (e.g., PKCS#7 padding) so data of arbitrary length can be processed in 128-bit blocks.
 - Accurately output encrypted binary ciphertext as hex-encoded or Base64 strings.
- **Constraints:**
 - Strictly **no cryptography libraries** (no `cryptography`, `pycryptodome`, `OpenSSL`, etc.).
 - All mathematical transformations, S-Box lookups/substitutions, matrix permutations, and byte-level operations must be written entirely from scratch.
- **Brownie Points:**
 - Implement **CBC (Cipher Block Chaining)** mode with Initialization Vectors (IV) alongside ECB.

4. Keylogger

- **What to build:** A local background keystroke monitor and activity logger designed for input diagnostics and typing analysis.
- **Core Features:**
 - Record typed input to a local `keystrokes.log` file with precise timestamps.
 - Translate and format non-printable special keys (e.g., `[SPACE]`, `[ENTER]`, `[BACKSPACE]`, `[SHIFT]`) rather than printing raw scan codes.
 - Include a clean exit hotkey sequence to stop recording, flush buffers, and display session stats (total keystrokes, active runtime).
- **Constraints:**
 - Strictly **no pre-built keylogger packages**.
 - If using Python, you may use standard low-level event hook libraries (like `pynput` or standard OS hooks) only to listen to hardware events, but all event handling, state mapping, buffer flushing, and file logging logic must be written by you.
- **Brownie Points:**
 - Real-time typing ergonomics metrics: Calculate average Words Per Minute (WPM) and backspace error rates.
 - Keystroke analytics summary: Display a frequency table or heatmap breakdown of the top 5 most pressed keys during the session.
 - Secure local storage: Encrypt the resulting diagnostic log file using your own cipher implementation from scratch.